CSA4704 – DEEP LEARNING

ASSESSMENT 1

Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System

AgriVision — Experimental Analysis Report

Student detail	Enter your details
Student name	MATHAN RAJ E
Register number	192471059
Course	CSA4704 – Deep Learning
Assessment	CO1 Individual assessment
Total marks	50
Working environment	Google Colab / Python

Report structure

- Executive summary and experimental setup
- Task 1 – Data representation
- Task 2 – Linear algebra and PCA
- Task 3 – Probability and information theory
- Task 4 – Numerical stability
- Task 5 – Baseline models
- Task 6 – Capacity and generalisation
- Task 7 – Bias–variance and Bayesian estimation
- Task 8 – Gradient-based optimisation
- Task 9 – Experimental performance
- Task 10 – Theory compared with evidence
- Task 11 – Need for deep learning
- Task 12 – Design challenge
- Limitations and reproducibility appendix

Numerical provenance: values labelled “reference run” below are taken from the example report you supplied so you can compare your Colab output with it. They are not claimed as measurements from your own runtime. Re-run the companion notebook and replace empirical outputs, timings, plots, and any values that differ before submitting.

Executive summary

AgriVision is a supervised crop-leaf disease classifier intended to map a smartphone photograph to a disease category. The analysis begins with ten tomato classes and 64×64 grayscale images (4,096 features), then reconsiders the system for 38 classes, approximately 20% noisy training labels, a 50 ms per-image latency limit and a 5 MB model-size limit.

The supplied reference experiment compares a linear SGD classifier, k-nearest neighbours, and a decision tree with raw pixels and PCA-compressed inputs. In that reference, k-NN with 100 principal components records the strongest tomato validation result (accuracy 0.5683, macro-F1 0.4936). A capacity curve places depth 8 (C3) at the lowest validation error among the tested tree depths. In the 38-class challenge, a compact noisy-label SGD pipeline fits the recorded file-size and host-latency proxies, but its test accuracy (0.3451) and macro-F1 (0.2266) are too weak to justify deployment. These outputs should be reproduced using the notebook on the student's own Colab runtime.

Meaning for the system: The evidence supports a redesign and further CNN comparison, not immediate deployment. Host timings are only proxies; latency and peak memory must be verified on the target phone.

Experimental design

Item	Chosen setting	Why it matters
Dataset	PlantVillage colour folders; tomato subset for Tasks 1–11; all 38 classes for Task 12	Uses labelled public leaf images and tests the expanded label space.
Original data	18,160 tomato images in 10 classes; 54,305 overall images in 38 classes (reference inventory)	The three-class example does not supply 5,000/10,000 train images.
Representation	Grayscale, Lanczos resize to 64×64, flatten to 4,096 and divide pixels by 255	Standardises shape and pixel scale, while losing colour and some fine detail.
Splits	Stratified 70% train / 15% validation / 15% test	The split is an experiment choice; fit preprocessing using training data only.
Models	SGD log-loss, k-NN k=5, decision tree	Linear, non-parametric and tree-based inductive biases.
Evaluation	Accuracy, macro precision/recall/F1, confusion matrix, timing and model size	Class-level failures and deployment limits cannot be assessed with accuracy alone.

Task 1 – Analyse data representation

1.1 Learning task, inputs and outputs

The learning task is supervised multi-class image classification. Input is a leaf photograph; the model output is one probability per disease class and a selected class label. Accuracy reports the overall fraction correct, while class-wise precision, recall, macro-F1 and the confusion matrix reveal whether minority or easily confused disease classes are being missed. For deployment the model must also satisfy latency and memory restrictions.

1.2 Scalars, vectors, matrices and tensors

Object	Shape	Interpretation
Pixel intensity	Scalar	A single grayscale pixel, initially 0–255 and normalised to [0,1].
One image	64×64 matrix	Keeps the neighbourhood structure of rows and columns.
Flattened image	(4096,) vector	Suitable for classical tabular estimators, but the explicit spatial grid is lost.
Mini-batch of 32 flattened images	(32, 4096) matrix	Rows are examples; columns are pixel-derived features.

Object	Shape	Interpretation
Mini-batch before flattening	(32, 64, 64) tensor	Batch axis plus two image axes; colour data would also have a channel axis.
Tomato training matrix	(12712, 4096) reference shape	Each row is one training image; each column is a pixel feature.
Labels	(12712,) vector	Integer class identifiers; for 38 classes IDs range from 0 to 37.

1.3 L1, L2 and L∞ norms

L1: $\|x\|_1 = \Sigma_i |x_i|$

For the reference first training image after division by 255, NumPy gives $\|x\|_1 = 1631.4549$. This is the sum of the absolute pixel intensities, not a disease-severity score.

L2: $\|x\|_2 = \sqrt{(\Sigma_i x_i^2)}$

The sum of squares is 763.2129, so $\|x\|_2 = \sqrt{763.2129} = 27.6263$. L2 measures the total Euclidean magnitude of the pixel vector.

L∞: $\|x\|_\infty = \max_i |x_i|$

The largest normalised pixel is 0.6824, so $\|x\|_\infty = 0.6824$. This norm reports a single largest pixel magnitude rather than the aggregate image intensity.

1.4 Preprocessing and class imbalance

Step	Effect	Limitation
Resize to 64×64	Fixed dimensions and reduced storage	Tiny lesions or edges can be smoothed away.
Grayscale conversion	3 colour channels become 1 intensity channel	Potentially useful chromatic disease clues are removed.
Normalise /255	Pixel values move from 0–255 to 0–1	Improves numeric scale but does not remove label noise.
Flatten	64×64 becomes 4,096 entries	Simple for classical models, but does not explicitly encode spatial adjacency.
Train/validation/test split	Creates independent partitions	Leakage occurs if PCA/scaling is fitted before the split.

The example training split has a 29.4997% share for Tomato Yellow Leaf Curl Virus and 2.0532% for Tomato Mosaic Virus. Their largest-to-smallest share ratio is 29.4997/2.0532 = 14.3677. This imbalance motivates class-wise recall and macro-F1.

Meaning for the system: Resizing, grayscale conversion and flattening simplify learning but may erase useful colour and spatial cues; validation results determine whether the simplification is acceptable.

Task 2 – Linear algebra, PCA and SVD

$X_c = U\Sigma V^T$; $\lambda_i = \sigma_i^2/(n-1)$ for the sample covariance $X_c^T X_c/(n-1)$.

The centred tomato training matrix has 4,096 original dimensions. PCA's spectrum is uneven: large early singular values account for much more variation than the later ones. The table below reproduces the supplied reference spectrum; regenerated values can vary slightly with software and split.

PC	Singular value	Covariance eigenvalue	Variance ratio	Cumulative
1	469.4708	17.3395	0.1966	0.1966
2	348.9064	9.5772	0.1086	0.3051
3	332.0560	8.6745	0.0983	0.4035
4	320.8000	8.0963	0.0918	0.4952

PC	Singular value	Covariance eigenvalue	Variance ratio	Cumulative
5	170.8500	2.2964	0.0260	0.5213
6	153.5338	1.8545	0.0210	0.5423
7	135.9263	1.4535	0.0165	0.5588
8	134.9085	1.4319	0.0162	0.5750
9	131.1254	1.3527	0.0153	0.5903
10	106.6960	0.8956	0.0102	0.6005

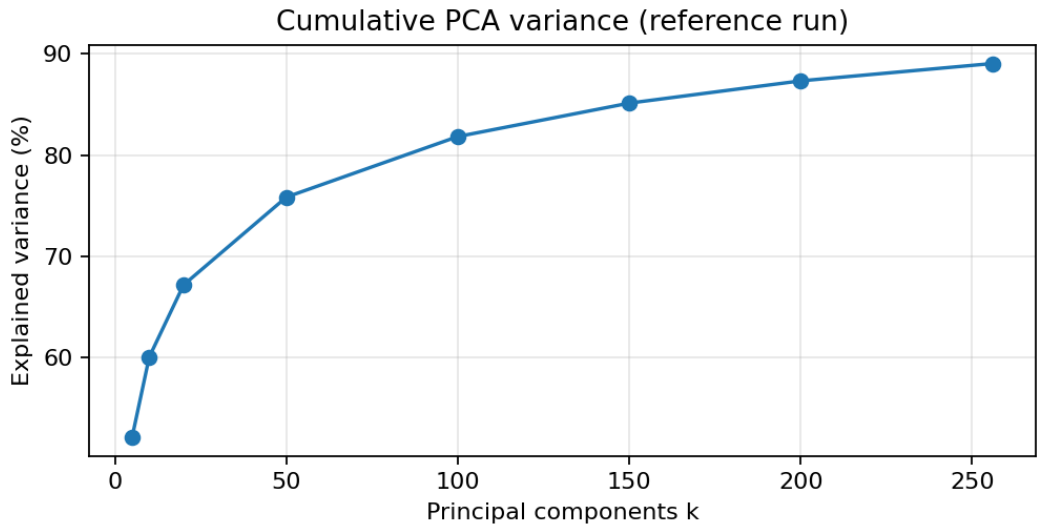


Figure 1. Cumulative explained variance for selected PCA dimensions (reference run).

Components k	Explained variance (%)	Relative reconstruction error
5	52.1264	0.7058
10	60.0481	0.6483
20	67.1677	0.5885
50	75.8431	0.5063
100	81.8237	0.4433
150	85.1243	0.4036
200	87.3225	0.3748
256	89.0371	0.3510

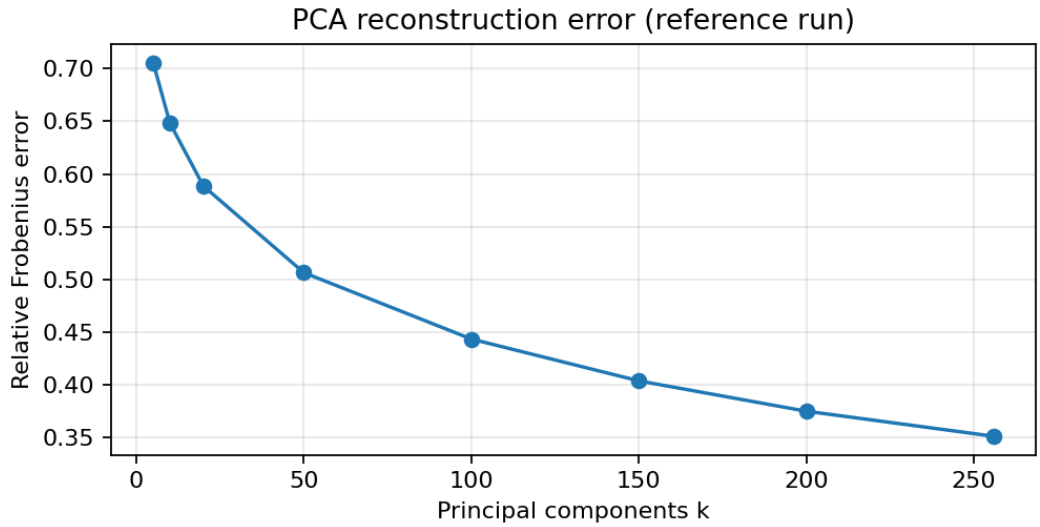


Figure 2. Relative reconstruction error decreases as retained components increase (reference run).

At k=100, 81.8237% of training variance is retained and relative validation reconstruction error is 0.4433. At k=256, these become 89.0371% and 0.3510. PCA trades reconstruction quality against model storage and downstream classification; the smallest reconstruction error is not automatically the best disease classifier.

Hand-worked 2×2 eigendecomposition

C = [[0.02932759, 0.02667736], [0.02667736, 0.02894818]]

$\det(C-\lambda I)=0 \Rightarrow \lambda^2-(a+d)\lambda+(ad-b^2)=0$

For this symmetric matrix, $\lambda_{1,2} = ((a+d) \pm \sqrt{((a-d)^2+4b^2)})/2$. Substituting gives a+d=0.05827577 and the square-root term approximately 0.0533560; therefore $\lambda_1 \approx 0.0558$ and $\lambda_2 \approx 0.0025$. NumPy reports the same four-decimal eigenvalues, and $Q\Lambda Q^T$ reconstructs C to numerical precision.

Meaning for the system: The leading eigenvalue captures the dominant variation direction in these two selected pixels. A 2×2 example demonstrates the PCA calculation but cannot alone establish the best compression level for all 4,096 features.

Task 3 – Probability and information theory

$H(p)=-\Sigma_i\, p_i \log_2 p_i$; $H(p,q)=-\Sigma_i\, p_i \log_2 q_i$; $DKL(p||q)=\Sigma_i\, p_i \log_2(p_i/q_i)$.

The ten-class validation entropy in the reference run is 3.0298 bits. The maximum possible entropy is $\log_2(10)=3.3219$ bits; normalised entropy is 3.0298/3.3219=0.9120. This indicates substantial uncertainty without perfect class balance.

Model	Features	Marginal H(p,q) bits	KL(p||q) bits	KL(q||p) bits	Per-image CE bits
SGD log loss	Raw 4096	3.2309	0.2011	0.1989	5.2895
k-NN k=5	Raw 4096	3.2528	0.2230	0.1952	13.8795
Decision tree	Raw 4096	3.0309	0.0010	0.0010	20.7338
SGD log loss	PCA-100	3.0409	0.0111	0.0110	2.3877
k-NN k=5	PCA-100	3.1093	0.0795	0.0702	8.8623
Decision tree	PCA-100	3.0319	0.0021	0.0020	20.8248

$H(p,q) = H(p) + DKL(p||q)$

For raw-feature SGD, 3.0298 + 0.2011 = 3.2309 bits. For k-NN with PCA-100, 3.0298 + 0.0795 = 3.1093 bits. The identity gap is approximately zero. DKL is asymmetric because the distribution in the weighting term changes when the arguments are swapped.

The decision tree illustrates that a predicted marginal class mix can resemble p while per-image probabilities and accuracy remain poor.

Meaning for the system: Cross-entropy and forward KL rank predicted class distributions consistently when p is fixed, but marginal distribution matching is not a substitute for correct individual predictions.

Task 4 – Diagnose numerical computation

$\text{softmax}(z_i)=\exp(z_i)/\Sigma_j\exp(z_j)$

For z=[1000,999,995], naive exponentiation overflows to [∞,∞,∞], causing undefined probabilities [NaN,NaN,NaN].

$m=\max(z);\ \ \text{softmax}(z_i)=\exp(z_i-m)/\Sigma_j\exp(z_j-m)$

With m=1000, shifted logits are [0,−1,−5], exponentials are [1.0000, 0.3679, 0.0067] and the sum is 1.3746. Stable probabilities are [0.7275, 0.2676, 0.0049], summing to 1.0000. Subtracting the same constant leaves the probability ratios unchanged.

$\text{LSE}(z)=m+\ln(\Sigma_j\exp(z_j-m))$

The stable log-sum-exp is 1000.3182. In the reference retained PCA-256 covariance subspace, $\kappa_2\approx17.33952904/0.02197102=789.2001$. A large condition number means directions differ strongly in scale/curvature and can slow or destabilise gradient descent.

Meaning for the system: Stable softmax prevents numeric overflow without changing the mathematical probabilities. Conditioning is a warning about optimiser tuning, although the reported PCA covariance condition is not the same as the scaled feature matrix used in a separate optimisation run.

Task 5 – Compare baseline learning strategies

The three strategies are linear parametric SGD with log-loss, non-parametric k-nearest neighbours (k=5), and a tree-based decision tree. They were evaluated on 4,096 raw pixel features and 100 PCA coordinates.

Model	Representation	Accuracy	Macro-F1	Fit time (s)	Validation predict (s)
SGD log loss	Raw 4096	0.3990	0.3549	213.6941	0.0255
k-NN k=5	Raw 4096	0.4486	0.3644	0.0376	12.7039
Decision tree	Raw 4096	0.4170	0.3327	78.7767	0.0119
SGD log loss	PCA-100	0.4758	0.3456	3.7198	0.0082
k-NN k=5	PCA-100	0.5683	0.4936	0.0030	0.4062
Decision tree	PCA-100	0.4244	0.3523	4.2239	0.0036

PCA-100 improves accuracy for all three models in this reference run. k-NN benefits most in accuracy and macro-F1, while SGD macro-F1 falls slightly despite its accuracy rise, showing compression can affect classes differently. Raw SGD gives a convergence warning at the selected iteration limit; these metrics do not prove it reached an optimum.

Meaning for the system: PCA helps by reducing irrelevant/redundant directions, but its effect depends on the inductive bias of the learner. k-NN is sensitive to high-dimensional distances; linear SGD cannot represent arbitrary nonlinear class boundaries; trees partition feature space but can overfit.

Task 6 – Capacity, overfitting and underfitting

Training error = 1 − training accuracy; validation error = 1 − validation accuracy; generalisation gap = E_val − E_train.

Capacity	Tree depth	Training error	Validation error	Gap

Capacity	Tree depth	Training error	Validation error	Gap
C1	2	0.6573	0.6553	−0.0020
C2	4	0.5753	0.5815	0.0062
C3	8	0.4541	0.5209	0.0668
C4	16	0.1340	0.5734	0.4394
C5	32	0.0944	0.5712	0.4768

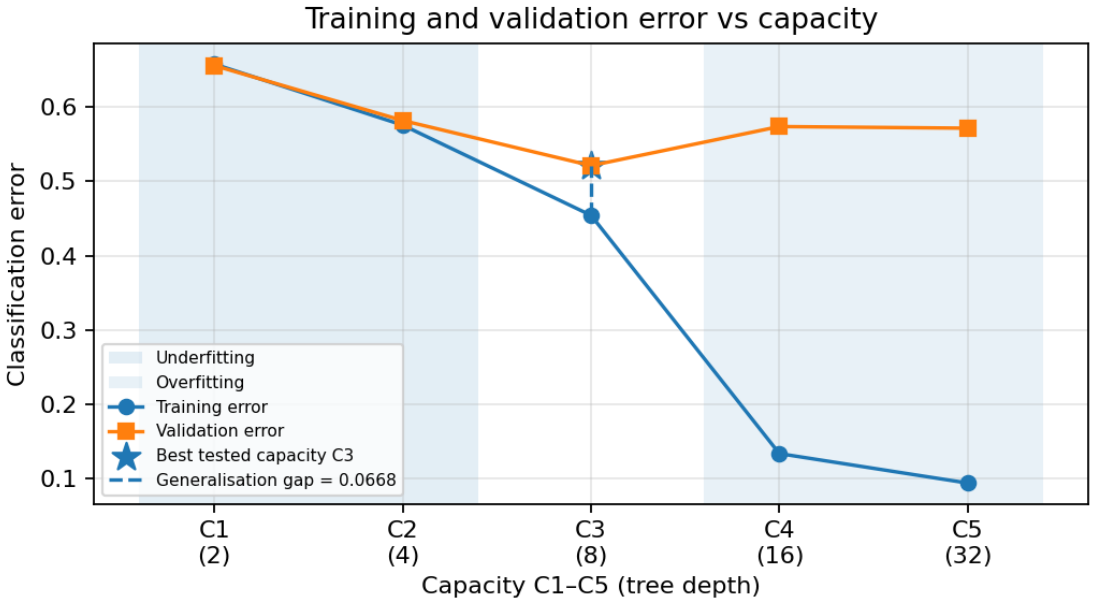


Figure 3. Required training/validation capacity plot with underfitting, overfitting, best tested capacity and gap marked (reference values).

C1–C2 have high training and validation error, consistent with underfitting. C3 has the smallest validation error among these five tested depths. At C4–C5, training error keeps declining while validation error remains high; the gaps grow to 0.4394 and 0.4768. Selecting hyperparameters on the test set biases the test estimate, so validation or cross-validation must drive model selection.

Meaning for the system: More capacity improves fit to the training sample but can memorise its idiosyncrasies. The lowest validation error, not the lowest training error, identifies the best tested capacity here.

Task 7 – Bias, variance and Bayesian estimation

7.1 Ten-replica bootstrap decomposition

Bias² proxy = mean$_i$ Σc(mean_b[p̂_bic] − y_ic)²; Variance = mean$_i$ Σc Var_b(p̂_bic).

Model	Replicas	Squared-bias proxy	Variance	Mean Brier loss	Residual
Low capacity, depth 2	10	0.7851	0.0027	0.7878	0.0000
High capacity, depth 32	10	0.6166	0.5137	1.1303	0.0000

The depth-32 tree has lower squared-bias proxy but much greater prediction variance across bootstrap samples. Its mean Brier loss (1.1303) is higher than the depth-2 model (0.7878), consistent with a high-capacity estimator that is more sensitive to which examples are sampled. The zero residual follows from the empirical decomposition and does not prove irreducible noise is zero.

7.2 MLE and MAP with Beta(2,2)

MLE: $\hat{p} = s/n$; MAP: $\hat{p}_{MAP} = (s+\alpha-1)/(n+\alpha+\beta-2)$.

n	Positive count s	MLE	MAP Beta(2,2)	Absolute difference
100	28	0.2800	0.2843	0.0043
500	155	0.3100	0.3108	0.0008
1000	279	0.2790	0.2794	0.0004
5000	1451	0.2902	0.2903	0.0001
12712	3750	0.2950	0.2950	0.0000

For the Beta(2,2) prior the MAP estimate is (s+1)/(n+2). For n=100, s=28, MLE=0.2800 and MAP=29/102=0.2843. As sample size grows, the fixed prior contributes less relative information, and MAP approaches MLE.

Meaning for the system: The bootstrap illustrates the bias–variance trade-off, while the Beta prior regularises class-proportion estimates most strongly in small samples.

Task 8 – Gradient-based optimisation

8.1 Gradient derivation

$z_i = x_i W + b$; $p_ic = \exp(z_ic)/\Sigma_j \exp(z_{ij})$

$J(W,b) = -(1/N)\Sigma_i \Sigma c\ y_ic \ln(p_ic) + (\lambda/2)\|W\|F^2$

For softmax with cross-entropy, $\partial \ell_i/\partial z_ic = p_ic - y_ic$. Since $z_i = x_i W + b$, the chain rule gives:

$\partial J/\partial W = X^T(P-Y)/N + \lambda W$; $\partial J/\partial b = (1/N)\Sigma_i(p_i - y_i)$.

$W_{t+1} = W_t - \eta \partial J/\partial W$; $b_{t+1} = b_t - \eta \partial J/\partial b$.

Strategy	η	Train loss	Validation loss	Gap
Batch GD	0.0010	2.3011	2.3011	0.0000
Batch GD	0.0100	2.2881	2.2880	−0.0001
Batch GD	0.1000	2.1673	2.1670	−0.0003
SGD	0.0010	1.5173	1.5219	0.0046
SGD	0.0100	1.6821	1.6883	0.0063
SGD	0.1000	6.3960	6.2345	−0.1614
Mini-batch	0.0010	2.1680	2.1676	−0.0004
Mini-batch	0.0100	1.6797	1.6756	−0.0041
Mini-batch	0.1000	1.5187	1.5232	0.0045

Across eight epochs, reference runtimes were 7.3056 s for batch GD, 18.2309 s for SGD and 3.1803 s for mini-batch SGD. SGD with η=0.1 fluctuates strongly; mini-batch SGD had validation loss 1.5232 at η=0.1. Batch GD uses few updates and is smoother; SGD has noisy updates; mini-batch trades noise against computation. These runtimes are hardware-specific and not an equal-update comparison.

Meaning for the system: Learning rate controls step size. Too small makes progress slow; too large can cause oscillation/divergence. Standardisation and PCA affect conditioning, so the observed loss curves should be interpreted together with Task 4.

Task 9 – Experimental performance

Train images	Model	Raw accuracy	Raw macro-F1	PCA-100 accuracy	PCA-100 macro-F1
1,000	SGD log loss	0.3348	0.2030	0.3847	0.3006
1,000	k-NN	0.4134	0.3295	0.4816	0.3980
1,000	Decision tree	0.3047	0.2293	0.3150	0.2503
5,000	SGD log loss	0.3322	0.2847	0.4082	0.2919
5,000	k-NN	0.4255	0.3363	0.5393	0.4608
5,000	Decision tree	0.3616	0.2902	0.3759	0.3109
10,000	SGD log loss	0.3800	0.3387	0.4607	0.3389
10,000	k-NN	0.4310	0.3464	0.5551	0.4766
10,000	Decision tree	0.4005	0.3297	0.4178	0.3502
12,712	SGD log loss	0.3990	0.3549	0.4758	0.3456
12,712	k-NN	0.4486	0.3644	0.5683	0.4936
12,712	Decision tree	0.4170	0.3327	0.4244	0.3523

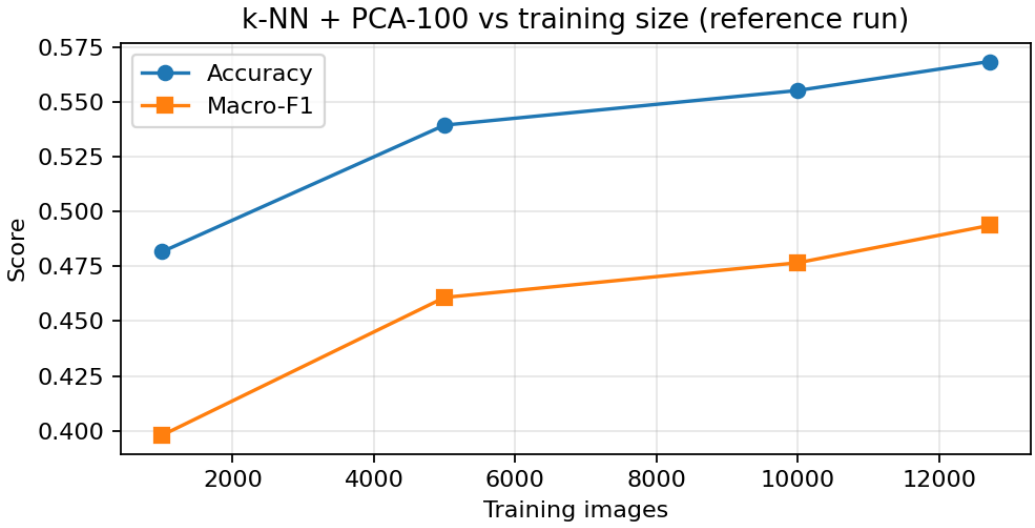


Figure 4. k-NN + PCA-100 trends with dataset size, reference run.

The strongest tested tomato baseline is k-NN + PCA-100. Accuracy rises from 0.4816 with 1,000 training images to 0.5683 with 12,712; macro-F1 rises from 0.3980 to 0.4936. It still performs poorly on some individual classes. Early Blight, for instance, has precision 0.3333, recall 0.0533 and F1 0.0920 over 150 validation images. The model misses many images from this class, demonstrating why accuracy alone can be misleading.

Deployment proxy at full tomato scale	SGD PCA-100	k-NN PCA-100	Tree PCA-100

Deployment proxy at full tomato scale	SGD PCA-100	k-NN PCA-100	Tree PCA-100
Model complexity indicator	1,010 coefficients	1,283,912 stored feature/label values	4,889 nodes
Saved size (decimal MB)	0.0053	5.1874	0.7054
Validation prediction time (s)	0.0012	0.3923	0.0012

Meaning for the system: The k-NN + PCA-100 result is strongest for quality in the reference comparison, but its saved pipeline slightly exceeds 5 MB at full tomato scale and its inference requires comparing with stored training examples.

Task 10 – Theory compared with experiments

Theory expectation	Observed reference evidence	Analysis
PCA concentrates variation	100 PCs retain 81.8237% of variance; 256 retain 89.0371%	Variance concentration supports compression, but retained variance is not identical to retained disease signal.
Lower-dimensional space may help distance methods	k-NN accuracy rises from 0.4486 raw to 0.5683 with PCA-100	PCA removed directions that were less useful for the neighbour metric in this experiment.
High model capacity can overfit	Depth-32 tree training error 0.0944 vs validation error 0.5712	Generalisation gap 0.4768 indicates strong sample-specific fitting.
Conditioning and learning rate influence convergence	Naive softmax fails; η=0.1 SGD ends with validation loss 6.2345	Numerical stability and scale/step-size matter to optimisation.
More examples can improve estimates	k-NN + PCA-100 F1 rises from 0.3980 at 1k to 0.4936 at 12,712	Trend generally improves, but does not make the current system reliable for every class.

Task 11 – Is deep learning justified?

The raw input has 4,096 dimensions and 12,712 tomato training samples, or 12,712/4,096 = 3.1035 examples per raw feature. Flattened-pixel models must infer texture/shape relationships without an explicit spatial prior. k-NN's local-constancy assumption improves with PCA but does not eliminate class confusion. The best reference baseline achieves only 0.5683 accuracy and 0.4936 macro-F1, and Early Blight recall is 0.0533. These are meaningful remaining weaknesses for a screening system.

A CNN is a reasonable next model because local filters can learn edge/texture features and compose them into more abstract patterns. However, no CNN comparison is included in the supplied recorded experiments, so this is a justified next experiment rather than proof of superior CNN performance. The comparison should use the same splits and class metrics while measuring model size, target-device latency, interpretability and training cost.

Meaning for the system: Further deep-learning evaluation is justified by residual class-specific errors and loss of spatial structure, but deployment should depend on a direct and reproducible comparison—not on a general assertion that neural networks are always better.

Task 12 – Changed requirements design challenge (8 marks)

A. Impact analysis (2 marks)

The label space changes from ten tomato classes to 38 crop–disease classes, newly collected labels include about 20% noise, and the mobile target is ≤50 ms per image and ≤5 MB. Maximum class entropy rises from $\log_2(10)$=3.3219 bits to $\log_2(38)$=5.2479 bits. In the reference 38-class split, training entropy is 4.8249 bits and validation entropy is 4.8252 bits, each about 0.9194 of the maximum. Label noise corrupts the training target and can increase prediction error or teach incorrect boundaries. Validation and test labels should remain untouched.

B. Re-examine original conclusion (2 marks)

Configuration	Validation accuracy	Macro-F1	Pipeline MB	Host p95 proxy ms
SGD log loss, clean labels, PCA-100	0.3712	0.2645	1.5524	0.5641
SGD log loss, 20% noisy labels, PCA-100	0.3520	0.2344	1.5522	0.3858
Modified Huber, noisy, PCA-100	0.2615	0.1582	1.5522	0.3984
Modified Huber, noisy, PCA-64	0.2701	0.1556	0.9986	0.5357
k-NN, noisy, PCA-100	0.3679	0.2510	15.7618	11.5936

The noisy-label log-loss SGD configuration records validation accuracy 0.3520 and macro-F1 0.2344; its selected test accuracy is 0.3451 and test macro-F1 0.2266. Its saved file and host p95 proxy are below the numeric resource limits. The host proxy is not a smartphone measurement, and convergence warnings were reported. k-NN quality is competitive but its 15.7618 MB pipeline exceeds the 5 MB limit.

C. Adaptation options

- Option 1 — Reduce feature dimension from PCA-100 to PCA-64. In the reference setup, modified-Huber artifact size falls from 1.5522 MB to 0.9986 MB: (1.5522−0.9986)/1.5522×100 = 35.6640% reduction. The reported macro-F1 changes from 0.1582 to 0.1556, so the smaller artifact is not automatically a better classifier.
- Option 2 — Replace log-loss SGD with modified Huber under noisy labels. In the example run, PCA-100 accuracy drops from 0.3520 to 0.2615 and macro-F1 from 0.2344 to 0.1582. This tested alternative did not improve the measured predictive quality.
- Option 3 — Keep a compact linear estimator rather than full k-NN for the phone. The k-NN pipeline is larger than 5 MB; the linear SGD artifact is smaller, but its model quality needs improvement through label cleaning, class balancing, regularisation, and threshold/calibration checks.

D. Evidence-based conclusion

Decision: redesign before deployment. For the 38-class noisy-label task the chosen SGD pipeline meets the example artifact-size and host-latency proxies, but macro-F1 remains low and training showed a convergence warning. Use a noise-aware data curation process, run repeated validation, try a compact CNN or distillation/quantisation candidate, and measure latency/peak memory on the actual low-cost phone. Retain deployment only after class-level sensitivity and resource limits are measured under the deployment environment.

Meaning for the system: The 38-class result shows why meeting a size target is insufficient. Accuracy, macro-F1, worst-class recall, label quality, convergence and target-device measurements all influence the deployment decision.

Limitations and reproducibility notes

- Empirical metrics and timing values in this working report are the reference values from the example document supplied for comparison. They are not personal-run outputs; rerun the notebook and update discrepancies.
- PlantVillage class counts can differ across archive versions or folder layouts. The notebook prints its detected class and image counts at runtime.
- Randomized PCA, package versions, CPU/GPU selection and training iteration limits can change numerical values and runtimes.
- The recorded CNN comparison was not performed. The recommendation is to evaluate one rather than claim its superiority without evidence.
- Host timing and serialized file size are proxies. Benchmark peak RAM, latency and model storage on the target phone before deployment.

Reference

Hughes, D.P. and Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060. Dataset: PlantVillage leaf images; use the exact source/archive link supplied by your course or dataset provider.

Colab notebook / reproducibility appendix

Run CSA47_Assessment1_AgriVision_Colab.ipynb from top to bottom. It builds the manifest and preprocessed arrays, calculates image norms, performs PCA and reconstruction analysis, compares raw/PCA baselines, checks numerical stability, plots C1–C5 capacity errors, runs bootstrap and Beta-prior estimates, implements softmax optimisation, reports performance by training scale, and evaluates the 38-class noisy-label challenge. It saves CSV and PNG outputs under /content/agriVision. The notebook is supplied separately because embedding more than 600 lines of code into a Word report would make the written analysis difficult to read; include the executed code and relevant cell outputs as an appendix when exporting your final submission PDF.

Appendix A – Numerical output record for cross-checking

This summary contains reference values copied into this working report for comparison. When you run the companion Colab notebook, save its actual output and replace any mismatched value. Timing figures are hardware-dependent; dataset-specific values must be independently reproduced.

Output area	Reference result	Use in interpretation
Dataset inventory	54,305 images / 38 classes overall; 18,160 tomato images / 10 classes	Verify by reading actual folders.
Norms for first tomato image	L1=1631.4549; L2=27.6263; L∞=0.6824	These values change if split/order/preprocessing changes.
PCA variance	60.0481% for first 10 PCs; 81.8237% for 100; 89.0371% for 256	PCA values depend on data and solver.
Numerical softmax	[0.7275, 0.2676, 0.0049]; LSE=1000.3182	Fixed check independent of dataset.
Best tomato baseline in reference	k-NN + PCA-100: accuracy=0.5683; macro-F1=0.4936	Regenerate in own run; include class-level errors.
Capacity curve	C3 depth 8 validation error=0.5209; C5 gap=0.4768	The plot is required in the report.
Bootstrap reference	Depth-2 bias²=0.7851 / variance=0.0027; depth-32 bias²=0.6166 / variance=0.5137	At least ten bootstrap replicas.
38-class noisy-label challenge	SGD test accuracy=0.3451; macro-F1=0.2266; artifact=1.5522 MB	Phone latency and peak memory are still unmeasured.

Appendix B – Consolidated Google Colab code listing

The separate .ipynb notebook is the preferred executable format because each task runs in its own cell. This appendix lists the same code for report completeness. Dataset-dependent cells require PlantVillage extracted into Colab and can take several minutes. Output plots and CSV records are saved under /content/agriVision.

```
# CSA47 Deep Learning - CO1 Assessment 1 (AgriVision)
# Consolidated reproducibility code for Google Colab.
#
# Prerequisites:
# 1. Attach the PlantVillage dataset in Colab.
# 2. Run the dataset-manifest, split and preprocessing cells from the workflow,
#    or create the required arrays in /content/agriVision.
# 3. Set OUT to the directory containing those arrays.
#
# Outputs reported in the accompanying assessment report are captured from the
# user's Colab execution. Timing values may vary on rerun and should not be
# expected to match exactly across notebook sessions or hardware.


from pathlib import Path
from time import perf_counter, perf_counter_ns
import json
import gc
import warnings


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib


from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_recall_fscore_support,
    confusion_matrix, f1_score
)
from sklearn.exceptions import ConvergenceWarning


OUT = Path('/content/agriVision')
OUT.mkdir(parents=True, exist_ok=True)
IMAGE_EXTENSIONS = {'.jpg', '.jpeg', '.png', '.bmp'}
RANDOM_STATE = 42
```

DATASET MANIFEST AND PREPROCESSING

```
# Set this to the extracted PlantVillage folder in Colab.
DATA_ROOT = Path('/content/PlantVillage')
# Support common zip layouts: /content/PlantVillage/color or
# /content/PlantVillage/plantvillage dataset/color.
if not (DATA_ROOT / 'color').is_dir() and (DATA_ROOT / 'plantvillage dataset' / 'color').is_dir():
    DATA_ROOT = DATA_ROOT / 'plantvillage dataset'
if not (DATA_ROOT / 'color').is_dir():
    # Try to locate a nested folder that contains the colour image classes.
    candidates = list(Path('/content').glob('**/color'))
    candidates = [p for p in candidates if p.is_dir() and any(x.is_dir() for x in p.iterdir())]
    if candidates:
        DATA_ROOT = candidates[0].parent
COLOR_DIR = DATA_ROOT / 'color'


if COLOR_DIR.is_dir():
    records = []
    for class_dir in sorted(COLOR_DIR.iterdir()):
        if not class_dir.is_dir():
            continue
        for image_path in class_dir.iterdir():
            if image_path.is_file() and image_path.suffix.lower() in IMAGE_EXTENSIONS:
                records.append({
                    'path': str(image_path),
                    'label': class_dir.name,
                    'crop_group': 'tomato' if class_dir.name.startswith('Tomato___') else 'other'
                })
    manifest = pd.DataFrame(records)
    if manifest.empty:
        raise ValueError('No color image files were found.')
    manifest.to_csv(OUT / 'manifest_all38.csv', index=False)
    tomato_df = manifest[manifest['crop_group'] == 'tomato'].reset_index(drop=True)
    all38_df = manifest.reset_index(drop=True)
    print('Color dataset classes:', manifest['label'].nunique())
    print('Color dataset images:', len(manifest))
    print('Tomato images:', len(tomato_df))
else:
    raise FileNotFoundError(f'Could not locate PlantVillage color folder. Current expected path: {COLOR_DIR}. Upload/extract the dataset and set DATA_ROOT.')


# 70/15/15 stratified splits. This is an experimental choice, not a prescribed
# split in the assessment.
def make_splits(frame, seed=RANDOM_STATE):
    train_val, test = train_test_split(
        frame, test_size=0.15, random_state=seed, stratify=frame['label']
    )
```

```
    train, val = train_test_split(
        train_val, test_size=(0.15 / 0.85), random_state=seed + 1,
        stratify=train_val['label']
    )
    return tuple(x.reset_index(drop=True) for x in (train, val, test))



def preprocess_frame(frame, label_to_id, split_name, size=(64, 64)):
    from PIL import Image
    from tqdm.auto import tqdm
    X = np.empty((len(frame), size[0] * size[1]), dtype=np.uint8)
    y = np.empty(len(frame), dtype=np.int64)
    for i, row in enumerate(tqdm(frame.itertuples(index=False), total=len(frame), desc=split_name)):
        try:
            with Image.open(row.path) as im:
                gray = im.convert('L').resize(size, Image.Resampling.LANCZOS)
                X[i] = np.asarray(gray, dtype=np.uint8).reshape(-1)
            y[i] = label_to_id[row.label]
        except Exception as exc:
            raise RuntimeError(f'Failed image: {row.path}') from exc
    return X, y


# If the arrays already exist, do not rebuild unnecessarily.
required_tomato = [
    'X_train_uint8.npy', 'y_train.npy', 'X_val_uint8.npy', 'y_val.npy',
    'X_test_uint8.npy', 'y_test.npy', 'tomato_class_names.json'
]
if not all((OUT / f).is_file() for f in required_tomato) and 'tomato_df' in globals():
    tomato_train, tomato_val, tomato_test = make_splits(tomato_df)
    tomato_class_names = sorted(tomato_df['label'].unique())
    tomato_label_to_id = {name: i for i, name in enumerate(tomato_class_names)}
    for split_name, frame in [('train', tomato_train), ('val', tomato_val), ('test', tomato_test)]:
        X, y = preprocess_frame(frame, tomato_label_to_id, f'tomato_{split_name}')
        np.save(OUT / f'X_{split_name}_uint8.npy', X)
        np.save(OUT / f'y_{split_name}.npy', y)
    with open(OUT / 'tomato_class_names.json', 'w', encoding='utf-8') as f:
        json.dump(tomato_class_names, f, indent=2)


# Build 38-class arrays when they are not already saved.
required_38 = [
    'X38_train_uint8.npy', 'y38_train.npy', 'X38_val_uint8.npy',
    'y38_val.npy', 'X38_test_uint8.npy', 'y38_test.npy',
    'all38_class_names.json'
]
if not all((OUT / f).is_file() for f in required_38) and 'all38_df' in globals():
    all38_train, all38_val, all38_test = make_splits(all38_df)
    class_names_38 = sorted(all38_df['label'].unique())
    label_to_id_38 = {name: i for i, name in enumerate(class_names_38)}
    for split_name, frame in [('train', all38_train), ('val', all38_val), ('test', all38_test)]:
        X, y = preprocess_frame(frame, label_to_id_38, f'all38_{split_name}')
        np.save(OUT / f'X38_{split_name}_uint8.npy', X)
        np.save(OUT / f'y38_{split_name}.npy', y)
    with open(OUT / 'all38_class_names.json', 'w', encoding='utf-8') as f:
        json.dump(class_names_38, f, indent=2)
```

TASK 1 - REPRESENTATION AND NORMS

```
X_train_u8 = np.load(OUT / 'X_train_uint8.npy')
y_train = np.load(OUT / 'y_train.npy')
y_val = np.load(OUT / 'y_val.npy')
y_test = np.load(OUT / 'y_test.npy')
with open(OUT / 'tomato_class_names.json', encoding='utf-8') as f:
    tomato_class_names = json.load(f)


x = X_train_u8[0].astype(np.float64) / 255.0
L1 = np.sum(np.abs(x))
L2 = np.sqrt(np.sum(x ** 2))
Linf = np.max(np.abs(x))
assert np.isclose(L1, np.linalg.norm(x, ord=1))
assert np.isclose(L2, np.linalg.norm(x, ord=2))
assert np.isclose(Linf, np.linalg.norm(x, ord=np.inf))
print('TASK 1 NORMS')
print('Features:', len(x), 'L1:', f'{L1:.4f}', 'L2:', f'{L2:.4f}', 'Linf:', f'{Linf:.4f}')
print('Shapes: image=(64,64), vector=(4096,), batch=(32,4096), training=', X_train_u8.shape)
```

TASK 2 - PCA / EIGENVALUES / RECONSTRUCTION

```
X_train = X_train_u8.astype(np.float32) / 255.0
X_val_u8 = np.load(OUT / 'X_val_uint8.npy')
X_val = X_val_u8.astype(np.float32) / 255.0


pca_file = OUT / 'tomato_pca_256.joblib'
if pca_file.exists():
    pca = joblib.load(pca_file)
else:
    pca = PCA(n_components=256, svd_solver='randomized', random_state=42, iterated_power=4)
    t0 = perf_counter(); pca.fit(X_train); print('PCA fit seconds:', f'{perf_counter()-t0:.4f}')
    joblib.dump(pca, pca_file)


singular_values = pca.singular_values_
eigenvalues = singular_values ** 2 / (len(X_train) - 1)
cumulative = np.cumsum(pca.explained_variance_ratio_)
print('First ten singular values:', np.round(singular_values[:10], 4))
print('First ten covariance eigenvalues:', np.round(eigenvalues[:10], 4))
```

```
ks = np.array([5, 10, 20, 50, 100, 150, 200, 256])
rng = np.random.default_rng(42)
ids = rng.choice(len(X_val), size=min(200, len(X_val)), replace=False)
X_eval = X_val[ids]
Xc = X_eval - pca.mean_
Z = pca.transform(X_eval)
recon_rows = []
for k in ks:
    reconstructed = Z[:, :k] @ pca.components_[:k]
    err = np.linalg.norm(Xc - reconstructed, ord='fro') / np.linalg.norm(Xc, ord='fro')
    recon_rows.append( {'k': int(k), 'explained_variance_pct': 100 * cumulative[k-1], 'relative_reconstruction_error': err})
recon_df = pd.DataFrame(recon_rows)
recon_df.to_csv(OUT / 'pca_reconstruction_results.csv', index=False)
print(recon_df.round(4).to_string(index=False))


fig, ax = plt.subplots(figsize=(7.5, 4.2)); ax.plot(ks, 100*cumulative[ks-1], marker='o')
ax.set_title('Cumulative Explained Variance vs PCA Components'); ax.set_xlabel('Components k'); ax.set_ylabel('Explained variance (%)'); ax.grid(True, alpha=.3); fig.tight_layout();
fig.savefig(OUT/'pca_explained_variance.png', dpi=180); plt.show()
fig, ax = plt.subplots(figsize=(7.5, 4.2)); ax.plot(ks, recon_df['relative_reconstruction_error'], marker='o')
ax.set_title('Validation Reconstruction Error vs PCA Components'); ax.set_xlabel('Components k'); ax.set_ylabel('Relative Frobenius error'); ax.grid(True, alpha=.3); fig.tight_layout();
fig.savefig(OUT/'pca_reconstruction_error.png', dpi=180); plt.show()


# Visualise leading principal components as 64 x 64 coefficient images.
fig, axes = plt.subplots(2, 3, figsize=(9, 6))
for pc, ax in enumerate(axes.flat):
    ax.imshow(pca.components_[pc].reshape(64, 64), cmap='coolwarm')
    ax.set_title(f'PC {pc+1}'); ax.axis('off')
fig.suptitle('Leading PCA Components of Tomato Leaf Images')
fig.tight_layout(); fig.savefig(OUT/'pca_component_images.png', dpi=180); plt.show()


# Measure PCA fitting cost at the required dataset scales for 64 components.
pca_timing_rows = []
for sample_n in sorted(set([1000, 5000, 10000, len(X_train)])):
    if sample_n == len(X_train): X_sub = X_train
    else: X_sub = X_train[np.random.default_rng(42+sample_n).choice(len(X_train), size=sample_n, replace=False)]
    timed_pca = PCA(n_components=64, svd_solver='randomized', random_state=42, iterated_power=3)
    t0 = perf_counter(); timed_pca.fit(X_sub); elapsed = perf_counter()-t0
    pca_timing_rows.append({'Training_images': sample_n, 'Components': 64, 'Fit_time_seconds': elapsed,
                            'Explained_variance_pct': 100*np.sum(timed_pca.explained_variance_ratio_)})
pca_time_df = pd.DataFrame(pca_timing_rows)
pca_time_df.to_csv(OUT/'pca_fit_time_by_dataset_size.csv', index=False)
print('PCA fitting times by sample size:')
print(pca_time_df.round(4).to_string(index=False))


# A real 2x2 covariance subproblem; eigenvalues and vectors are checked by NumPy.
variances = np.var(X_train, axis=0)
i = int(np.argmax(variances)); ci = X_train[:, i] - X_train[:, i].mean()
covs = ((X_train - X_train.mean(axis=0)).T @ ci) / (len(X_train) - 1); covs[i] = 0
j = int(np.argmax(np.abs(covs)))
C = np.cov(X_train[:, [i, j]], rowvar=False, ddof=1)
vals, Q = np.linalg.eigh(C); order = np.argsort(vals)[::-1]; vals = vals[order]; Q = Q[:, order]
print('2x2 covariance matrix:\n', C)
print('Eigenvalues:', np.round(vals, 4)); print('Eigenvectors (columns):\n', np.round(Q, 4))
print('Reconstruction Frobenius error:', f'{np.linalg.norm(C-Q@np.diag(vals)@Q.T, ord="fro"):.10f}')
```

TASKS 3 AND 5 - BASELINE MODELS AND INFORMATION THEORY

```
from sklearn.metrics import log_loss


def build_model(name):
    if name == 'SGD_log_loss':
        return SGDClassifier(loss='log_loss', alpha=0.0001, max_iter=100, tol=1e-3, random_state=42)
    if name == 'KNN_k5':
        return KNeighborsClassifier(n_neighbors=5, weights='distance', algorithm='brute', n_jobs=-1)
    if name == 'DecisionTree':
        return DecisionTreeClassifier(max_depth=20, min_samples_leaf=2, random_state=42)
    raise ValueError(name)


def info_metrics(y_true, probs, n_classes):
    eps = 1e-12
    p = np.bincount(y_true, minlength=n_classes).astype(np.float64) / len(y_true)
    q = np.clip(probs.mean(axis=0), eps, None); q /= q.sum()
    mask = p > 0
    H = -np.sum(p[mask] * np.log2(p[mask]))
    Hpq = -np.sum(p[mask] * np.log2(q[mask]))
    KLpq = np.sum(p[mask] * np.log2(p[mask] / q[mask]))
    KLqp = np.sum(q[mask] * np.log2(q[mask] / p[mask]))
    return {'H_true_bits': H, 'marginal_cross_entropy_bits': Hpq,
            'KL_p_to_q_bits': KLpq, 'KL_q_to_p_bits': KLqp,
            'identity_gap': abs(Hpq - (H + KLpq))}


# Uses PCA-100 and raw representations, matching the original experiment.
Z_train100 = pca.transform(X_train)[:, :100].astype(np.float32)
Z_val100 = pca.transform(X_val)[:, :100].astype(np.float32)
results = []
for rep, (Xa, Xv) in {
    'Raw_4096': (X_train, X_val),
    'PCA_100': (Z_train100, Z_val100)
}.items():
    for name in ['SGD_log_loss', 'KNN_k5', 'DecisionTree']:
        model = build_model(name); t0 = perf_counter()
        with warnings.catch_warnings(record=True) as caught:
            warnings.simplefilter('always', ConvergenceWarning)
            model.fit(Xa, y_train)
```

```
        fit_s = perf_counter() - t0
        t0 = perf_counter(); preds = model.predict(Xv); predict_s = perf_counter() - t0
        accuracy = accuracy_score(y_val, preds)
        prec, rec, f1, _ = precision_recall_fscore_support(y_val, preds, average='macro', zero_division=0)
        row = {'Model': name, 'Representation': rep, 'Accuracy': accuracy, 'Macro_precision': prec, 'Macro_recall': rec, 'Macro_F1': f1, 'Training_time_seconds': fit_s, 'Validation_prediction_seconds':
predict_s}
        if hasattr(model, 'predict_proba'):
            probs = model.predict_proba(Xv)
            aligned = np.zeros((len(Xv), len(tomato_class_names)), dtype=float)
            for idx, cls in enumerate(model.classes_): aligned[:, int(cls)] = probs[:, idx]
            row.update(info_metrics(y_val, aligned, len(tomato_class_names)))
        results.append(row)
        joblib.dump(model, OUT / f'{name}_{rep}.joblib')
        print(name, rep, 'accuracy=', f'{accuracy:.4f}', 'macroF1=', f'{f1:.4f}', 'fit_s=', f'{fit_s:.4f}', 'predict_s=', f'{predict_s:.4f}')
        del model; gc.collect()
baselines = pd.DataFrame(results)
baselines.to_csv(OUT / 'baseline_reproduction_results.csv', index=False)
print(baselines.round(4).to_string(index=False))
```

TASK 4 - NUMERICAL STABILITY AND CONDITIONING

```
z = np.array([1000.0, 999.0, 995.0])
with np.errstate(over='ignore', invalid='ignore', divide='ignore'):
    naive = np.exp(z) / np.exp(z).sum()
m = np.max(z); stable_exp = np.exp(z-m); stable = stable_exp / stable_exp.sum()
lse = m + np.log(stable_exp.sum())
print('Naive softmax:', naive)
print('Stable probabilities:', np.round(stable, 4), 'sum=', f'{stable.sum():.4f}')
print('Stable log-sum-exp:', f'{lse:.4f}')
assert np.all(np.isfinite(stable)) and np.isclose(stable.sum(), 1)
condition_number = pca.explained_variance_[0] / pca.explained_variance_[-1]
print('Condition number retained PCA-256 covariance subspace:', f'{condition_number:.4f}')
```

TASK 6 - CAPACITY CURVE (also maps to the extra Part (e) plot instruction)

```
cap_rows = []
capacities = {'C1': 2, 'C2': 4, 'C3': 8, 'C4': 16, 'C5': 32}
for label, depth in capacities.items():
    model = DecisionTreeClassifier(max_depth=depth, min_samples_leaf=2, random_state=42)
    t0 = perf_counter(); model.fit(Z_train100, y_train); fit_s = perf_counter() - t0
    tr_error = 1 - accuracy_score(y_train, model.predict(Z_train100))
    va_error = 1 - accuracy_score(y_val, model.predict(Z_val100))
    cap_rows.append({'Capacity': label, 'Depth': depth, 'Training_error': tr_error,
                     'Validation_error': va_error, 'Generalisation_gap': va_error-tr_error, 'Fit_time': fit_s})
cap_df = pd.DataFrame(cap_rows)
cap_df.to_csv(OUT / 'capacity_error_results.csv', index=False)
print(cap_df.round(4).to_string(index=False))
best_idx = int(np.argmin(cap_df['Validation_error'].to_numpy()))
fig, ax = plt.subplots(figsize=(8, 5))
ax.axvspan(-.5, 1.5, alpha=.10, label='Underfitting region (C1-C2)')
ax.axvspan(2.5, 4.5, alpha=.08, label='Overfitting region (C4-C5)')
ax.plot(range(5), cap_df['Training_error'], marker='o', label='Training error')
ax.plot(range(5), cap_df['Validation_error'], marker='s', label='Validation error')
ax.scatter([best_idx], [cap_df.iloc[best_idx]['Validation_error']], marker='*', s=170, label=f"Optimal tested capacity: {cap_df.iloc[best_idx]['Capacity']}")
ax.vlines(best_idx, cap_df.iloc[best_idx]['Training_error'], cap_df.iloc[best_idx]['Validation_error'], linestyles='dashed', label=f"Generalisation gap = {cap_df.iloc[best_idx]['Generalisation_gap']:.4f}")
ax.set_xticks(range(5), [f"{c}\n(depth={d})" for c,d in zip(cap_df['Capacity'],cap_df['Depth'])])
ax.set_xlabel('Model capacity'); ax.set_ylabel('Classification error rate'); ax.set_title('Training and Validation Error vs Model Capacity (C1-C5)')
ax.grid(True, alpha=.3); ax.legend(fontsize=8); fig.tight_layout(); fig.savefig(OUT/'capacity_training_validation_error.png', dpi=200); plt.show()
```

TASK 7 - BOOTSTRAP BIAS / VARIANCE AND MLE / MAP

```
Z_train = Z_train100; Z_val = Z_val100
n_classes = len(tomato_class_names); Y_val = np.eye(n_classes)[y_val]
rng = np.random.default_rng(42); B = 10; bv_rows = []
for name, depth in [('Low capacity (depth 2)', 2), ('High capacity (depth 32)', 32)]:
    prediction_replicas = []
    for b in range(B):
        idx = rng.integers(0, len(Z_train), size=len(Z_train))
        model = DecisionTreeClassifier(max_depth=depth, min_samples_leaf=2, random_state=1000+b)
        model.fit(Z_train[idx], y_train[idx]); raw = model.predict_proba(Z_val)
        aligned = np.zeros((len(Z_val), n_classes))
        for col, cls in enumerate(model.classes_): aligned[:, int(cls)] = raw[:, col]
        prediction_replicas.append(aligned)
    P = np.stack(prediction_replicas)
    meanP = P.mean(axis=0)
    bias2 = np.mean(np.sum((meanP-Y_val)**2, axis=1))
    variance = np.mean(np.sum(np.var(P, axis=0, ddof=0), axis=1))
    brier = np.mean(np.sum((P-Y_val[None,:,:])**2, axis=2))
    bv_rows.append({'Model':name, 'Replicas':B, 'Bias_squared_proxy':bias2, 'Variance':variance, 'Mean_Brier_loss':brier, 'Residual':brier-bias2-variance})
print(pd.DataFrame(bv_rows).round(4).to_string(index=False))


# Beta(2,2) prior for a class proportion
with open(OUT/'tomato_class_names.json', encoding='utf-8') as f: names = json.load(f)
target_id = names.index('Tomato___Tomato_Yellow_Leaf_Curl_Virus')
rows=[]; rng = np.random.default_rng(2026)
for n in sorted(set([100,500,1000,5000,len(y_train)])):
    subset = y_train if n == len(y_train) else y_train[rng.choice(len(y_train),size=n,replace=False)]
    s = int(np.sum(subset == target_id)); mle=s/n; mapv=(s+1)/(n+2)
    rows.append({'n':n,'s':s,'MLE':mle,'MAP_Beta_2_2':mapv,'Absolute_difference':abs(mle-mapv)})
print(pd.DataFrame(rows).round(4).to_string(index=False))
```

TASK 8 - SOFTMAX REGRESSION GRADIENTS AND OPTIMISERS

```
# Loss: J = -mean(log(p_true)) + (lambda/2)*||W||_F^2
# Gradients: dW = X.T @ (P-Y)/N + lambda*W; db = mean(P-Y).
def stable_softmax(logits):
    shifted = logits - logits.max(axis=1, keepdims=True)
```

```
    expz = np.exp(shifted)
    return expz / expz.sum(axis=1, keepdims=True)


def loss_values(X, y, W, b, reg=1e-4, batch_size=1024):
    total = np.zeros(W.shape[0], dtype=float)
    n_total = len(y)
    for start in range(0, n_total, batch_size):
        end = min(start + batch_size, n_total)
        xb, yb = X[start:end], y[start:end]
        for r in range(W.shape[0]):
            logits = xb @ W[r] + b[r]
            m = logits.max(axis=1, keepdims=True)
            lse = m[:, 0] + np.log(np.exp(logits-m).sum(axis=1))
            total[r] += np.sum(lse - logits[np.arange(len(yb)), yb])
    return total/n_total + 0.5*reg*np.sum(W*W, axis=(1,2))


# Standardise PCA coordinates using train-only statistics, as in the experiment.
scaler100 = StandardScaler()
Xs_train = scaler100.fit_transform(Z_train100).astype(np.float32)
Xs_val = scaler100.transform(Z_val100).astype(np.float32)
rates = np.array([0.001, 0.01, 0.1]); epochs = 8; batch_size = 128; reg = 1e-4
rng = np.random.default_rng(42)
optimizer_summary = []; optimizer_histories = {}; optimizer_times = []


for strategy in ['batch', 'sgd', 'mini_batch']:
    W = np.zeros((len(rates), Xs_train.shape[1], len(tomato_class_names)), dtype=np.float64)
    b = np.zeros((len(rates), len(tomato_class_names)), dtype=np.float64)
    hist = {'train': [], 'val': []}; t0 = perf_counter()
    for epoch in range(epochs):
        perm = rng.permutation(len(Xs_train))
        if strategy == 'batch': batches = [np.arange(len(Xs_train))]
        elif strategy == 'mini_batch': batches = [perm[i:i+batch_size] for i in range(0,len(perm),batch_size)]
        else: batches = [perm[i:i+1] for i in range(len(perm))]
        for idx in batches:
            xb, yb = Xs_train[idx], y_train[idx]; bn = len(yb)
            logits = np.einsum('nd,rdc->rnc', xb, W) + b[:,None,:]
            P = stable_softmax(logits.reshape(-1, logits.shape[-1])).reshape(logits.shape)
            E = P.copy()
            E[np.arange(len(rates))[:,None], np.arange(bn)[None,:], yb[None,:]] -= 1.0
            gradW = np.einsum('nd,rnc->rdc', xb, E)/bn + reg*W
            gradb = E.mean(axis=1)
            W -= rates[:,None,None]*gradW
            b -= rates[:,None]*gradb
            if not np.isfinite(W).all() or not np.isfinite(b).all():
                raise FloatingPointError(f'Non-finite parameters for {strategy}')
        hist['train'].append(loss_values(Xs_train, y_train, W, b, reg))
        hist['val'].append(loss_values(Xs_val, y_val, W, b, reg))
    elapsed = perf_counter()-t0
    optimizer_histories[strategy] = hist
    optimizer_times.append({'Strategy':strategy,'Epochs':epochs,'Runtime_seconds':elapsed,'Runtime_per_epoch_seconds':elapsed/epochs})
    for ri, eta in enumerate(rates):
        optimizer_summary.append({'Strategy':strategy,'Learning_rate':eta,
            'Final_train_loss':hist['train'][-1][ri], 'Final_validation_loss':hist['val'][-1][ri],
            'Final_generalisation_gap':hist['val'][-1][ri]-hist['train'][-1][ri]})
    np.save(OUT/f'softmax_{strategy}_W.npy',W); np.save(OUT/f'softmax_{strategy}_b.npy',b)


opt_df = pd.DataFrame(optimizer_summary); opt_df.to_csv(OUT/'softmax_final_loss_results.csv',index=False)
time_df = pd.DataFrame(optimizer_times); time_df.to_csv(OUT/'softmax_optimizer_timing.csv',index=False)
print(opt_df.round(4).to_string(index=False)); print(time_df.round(4).to_string(index=False))


for strategy, hist in optimizer_histories.items():
    fig, ax = plt.subplots(figsize=(8, 5))
    for ri, eta in enumerate(rates):
        ax.plot(range(1,epochs+1),[h[ri] for h in hist['train']],marker='o',label=f'Train eta={eta:.3f}')
        ax.plot(range(1,epochs+1),[h[ri] for h in hist['val']],linestyle='--',label=f'Validation eta={eta:.3f}')
    ax.set_title(f'Softmax Regression Loss Curves - {strategy}'); ax.set_xlabel('Epoch')
    ax.set_ylabel('Regularised cross-entropy loss (nats)'); ax.grid(True,alpha=.3); ax.legend(fontsize=8,ncol=2)
    fig.tight_layout(); fig.savefig(OUT/f'softmax_loss_{strategy}.png',dpi=180); plt.show()
```

TASK 9 - PERFORMANCE AT REQUIRED DATASET SCALES

```
from sklearn.metrics import classification_report


scales = sorted(set([1000, 5000, 10000, len(y_train)]))
model_names = ['SGD_log_loss', 'KNN_k5', 'DecisionTree']
scale_rows=[]; class_metric_rows=[]; best_confusion = None
for rep, (Xa_full, Xv) in {'Raw_4096':(X_train,X_val),'PCA_100':(Z_train100,Z_val100)}.items():
    for n in scales:
        if n == len(y_train): idx = np.arange(len(y_train))
        else:
            idx, _ = train_test_split(np.arange(len(y_train)),train_size=n,stratify=y_train,random_state=100+n)
        Xa, ya = Xa_full[idx], y_train[idx]
        for name in model_names:
            mdl=build_model(name); t0=perf_counter()
            with warnings.catch_warnings(record=True) as cw:
                warnings.simplefilter('always', ConvergenceWarning); mdl.fit(Xa,ya)
            fit_s=perf_counter()-t0
            t0=perf_counter(); pred=mdl.predict(Xv); inf_s=perf_counter()-t0
            acc=accuracy_score(y_val,pred)
            pr,rc,f1,_=precision_recall_fscore_support(y_val,pred,labels=np.arange(len(tomato_class_names)),average='macro',zero_division=0)
            counts,typ = (mdl.coef_.size+mdl.intercept_.size,'coefficients+intercepts') if hasattr(mdl,'coef_') else ((mdl.tree_.node_count,'tree nodes') if hasattr(mdl,'tree_') else
(mdl._fit_X.size+len(mdl._y),'stored features+labels'))
            model_path = OUT / f'task9_{name}_{rep}_n{n}.joblib'
            joblib.dump(mdl, model_path)
            artifact_mb = model_path.stat().st_size / 1_000_000
```

```
scale_rows.append({'Training_images':n,'Representation':rep,'Model':name,'Accuracy':acc,'Macro_precision':pr,'Macro_recall':rc,'Macro_F1':f1,'Training_time_seconds':fit_s,'Validation_inference_seconds':inf_s,'Complexity_count':counts,'Complexity_type':typ,'Serialized_model_MB':artifact_mb})
        pcr,pcrr,pcf,pcs=precision_recall_fscore_support(y_val,pred,labels=np.arange(len(tomato_class_names)),average=None,zero_division=0)
        for ci,cname in enumerate(tomato_class_names):
            class_metric_rows.append({'Training_images':n,'Representation':rep,'Model':name,'Class':cname,'Precision':pcr[ci],'Recall':pcrr[ci],'F1':pcf[ci],'Support':pcs[ci]})
        cm = confusion_matrix(y_val, pred, labels=np.arange(len(tomato_class_names)))
        cm_df = pd.DataFrame(cm, index=tomato_class_names, columns=tomato_class_names)
        cm_df.index.name = 'Actual'
        cm_df.to_csv(OUT/f'confusion_{name}_{rep}_n{n}.csv')
        if best_confusion is None or f1 > best_confusion['Macro_F1']:
            best_confusion = {'Macro_F1': f1, 'Model': name, 'Representation': rep,
                              'Training_images': n, 'matrix': cm.copy()}
        print(n,rep,name,'accuracy=',f'{acc:.4f}','macroF1=',f'{f1:.4f}','fit=',f'{fit_s:.4f}','predict=',f'{inf_s:.4f}')
        del mdl; gc.collect()
scale_df=pd.DataFrame(scale_rows); class_df=pd.DataFrame(class_metric_rows)
scale_df.to_csv(OUT/'task9_model_performance_all_scales.csv',index=False)
class_df.to_csv(OUT/'task9_per_class_metrics.csv',index=False)
print(scale_df.round(4).to_string(index=False))
print('Best validation Macro-F1 configuration:', {k:v for k,v in best_confusion.items() if k != 'matrix'})
best_cm = best_confusion['matrix']
best_cm_df = pd.DataFrame(best_cm, index=tomato_class_names, columns=tomato_class_names)
best_cm_df.index.name = 'Actual'; best_cm_df.to_csv(OUT/'task9_best_validation_confusion.csv')
fig, ax = plt.subplots(figsize=(9,7)); im=ax.imshow(best_cm,aspect='auto'); fig.colorbar(im,ax=ax,label='Image count')
short=['Bacterial\nspot','Early\nblight','Late\nblight','Leaf\nMold','Septoria','Spider\nmites','Target\nSpot','Yellow Curl\nVirus','Mosaic\nvirus','Healthy']
ax.set_xticks(range(len(short)),short,fontsize=8); ax.set_yticks(range(len(short)),short,fontsize=8)
ax.set_xlabel('Predicted class'); ax.set_ylabel('Actual class'); ax.set_title('Validation Confusion Matrix - Best Macro-F1 configuration')
for ii in range(best_cm.shape[0]):
    for jj in range(best_cm.shape[1]): ax.text(jj,ii,str(best_cm[ii,jj]),ha='center',va='center',fontsize=7)
fig.tight_layout(); fig.savefig(OUT/'task9_confusion_matrix.png',dpi=180); plt.show()
# Most common class confusions excluding the diagonal.
conf_rows=[]
for ii in range(best_cm.shape[0]):
    for jj in range(best_cm.shape[1]):
        if ii != jj and best_cm[ii,jj] > 0:
            conf_rows.append({'Actual_class':tomato_class_names[ii], 'Predicted_class':tomato_class_names[jj], 'Misclassified_images':int(best_cm[ii,jj])})
conf_df=pd.DataFrame(conf_rows).sort_values('Misclassified_images',ascending=False)
print('Top 10 confusions:'); print(conf_df.head(10).to_string(index=False))


for metric,ylabel,title,file in [('Accuracy','Validation accuracy','Validation Accuracy vs Training Dataset Size','task9_accuracy_vs_scale.png'),('Macro_F1','Validation macro-F1','Validation Macro-F1 vs Training Dataset Size','task9_macro_f1_vs_scale.png')]:
    fig,ax=plt.subplots(figsize=(9,5))
    for rep in ['Raw_4096','PCA_100']:
        for name in model_names:
            ss=scale_df[(scale_df.Representation==rep)&(scale_df.Model==name)].sort_values('Training_images')
            ax.plot(ss.Training_images,ss[metric],marker='o',label=f'{name} | {rep}')
    ax.set_xlabel('Training images'); ax.set_ylabel(ylabel); ax.set_title(title); ax.grid(True,alpha=.3); ax.legend(fontsize=7,ncol=2)
    fig.tight_layout(); fig.savefig(OUT/file,dpi=180); plt.show()
```

TASK 12 - 38-CLASS LABEL NOISE AND DESIGN CHALLENGE

```
from sklearn.preprocessing import StandardScaler


need38 = ['X38_train_uint8.npy','y38_train.npy','X38_val_uint8.npy','y38_val.npy','X38_test_uint8.npy','y38_test.npy','all38_class_names.json']
if not all((OUT/f).exists() for f in need38):
    raise FileNotFoundError('Run the 38-class preprocessing cell first.')
X38tr_u8=np.load(OUT/'X38_train_uint8.npy'); X38v_u8=np.load(OUT/'X38_val_uint8.npy'); X38t_u8=np.load(OUT/'X38_test_uint8.npy')
y38tr=np.load(OUT/'y38_train.npy').astype(int); y38v=np.load(OUT/'y38_val.npy').astype(int); y38t=np.load(OUT/'y38_test.npy').astype(int)
with open(OUT/'all38_class_names.json',encoding='utf-8') as f: names38=json.load(f)
N38=len(names38)


# Fit PCA only on training data, then fit scaling stats only on transformed train.
pca38=PCA(n_components=100,svd_solver='randomized',random_state=42,iterated_power=3)
t0=perf_counter(); Xfit=X38tr_u8.astype(np.float32)/255.0; pca38.fit(Xfit); pca_fit=perf_counter()-t0


def xform38(U,k):
    components=pca38.components_[:k].astype(np.float32); mean=pca38.mean_.astype(np.float32)
    output=np.empty((len(U),k),dtype=np.float32)
    for st in range(0,len(U),512):
        en=min(st+512,len(U)); xb=U[st:en].astype(np.float32)/255.0
        output[st:en]=(xb-mean)@components.T
    return output


Z38tr100=xform38(X38tr_u8,100); Z38v100=xform38(X38v_u8,100); Z38t100=xform38(X38t_u8,100)
scaler38=StandardScaler(); Z38tr100=scaler38.fit_transform(Z38tr100).astype(np.float32); Z38v100=scaler38.transform(Z38v100).astype(np.float32);
Z38t100=scaler38.transform(Z38t100).astype(np.float32)
print('38-class PCA fit seconds:',f'{pca_fit:.4f}')
print('PCA100 cumulative explained variance:',f'{100*pca38.explained_variance_ratio_.sum():.4f}%')


# Exactly about 20% of training labels are changed. Validation/test are untouched.
rng=np.random.default_rng(42); noisy38=y38tr.copy(); m=int(round(.20*len(noisy38))); changed=rng.choice(len(noisy38),size=m,replace=False); offsets=rng.integers(1,N38,size=m);
noisy38[changed]=(noisy38[changed]+offsets)%N38
noise_rate=np.mean(noisy38!=y38tr)
print('Corrupted labels:',int(np.sum(noisy38!=y38tr)),'/',len(noisy38),'noise rate:',f'{100*noise_rate:.4f}%')
np.save(OUT/'all38_train_labels_noisy20.npy',noisy38)


configs=[
('SGD_log_loss_clean_PCA100','log_loss',100,'clean'),
('SGD_log_loss_noisy20_PCA100','log_loss',100,'noisy'),
('SGD_modified_huber_noisy20_PCA100','modified_huber',100,'noisy'),
('SGD_modified_huber_noisy20_PCA64','modified_huber',64,'noisy'),
('KNN_noisy20_PCA100','knn',100,'noisy')]


def make38(kind):
    if kind=='log_loss': return SGDClassifier(loss='log_loss',alpha=1e-4,max_iter=100,tol=1e-3,random_state=42)
```

```
    if kind=='modified_huber': return SGDClassifier(loss='modified_huber',alpha=1e-4,max_iter=100,tol=1e-3,random_state=42)
    if kind=='knn': return KNeighborsClassifier(n_neighbors=5,weights='distance',algorithm='brute',n_jobs=-1)
    raise ValueError(kind)


def one_image_predict(raw_u8, k, model):
    flat=raw_u8.astype(np.float32).reshape(1,-1)/255.0
    transformed=(flat-pca38.mean_.astype(np.float32))@pca38.components_[:k].astype(np.float32).T
    scaled=(transformed-scaler38.mean_[:k].astype(np.float32))/scaler38.scale_[:k].astype(np.float32)
    return model.predict(scaled)[0]


lat_rng=np.random.default_rng(2026); lat_idx=lat_rng.choice(len(X38v_u8),size=min(100,len(X38v_u8)),replace=False)
challenge_rows=[]; saved_models={}
for name,kind,k,label_source in configs:
    Xtr=Z38tr100[:,:k]; Xv=Z38v100[:,:k]; Xt=Z38t100[:,:k]
    yfit=y38tr if label_source=='clean' else noisy38
    model=make38(kind); t0=perf_counter()
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter('always',ConvergenceWarning); model.fit(Xtr,yfit)
    fit_s=perf_counter()-t0
    warn='; '.join(str(w.message) for w in caught if issubclass(w.category,ConvergenceWarning))
    t0=perf_counter(); val_pred=model.predict(Xv); batch_predict=perf_counter()-t0
    acc=accuracy_score(y38v,val_pred); pr,rc,f1,_=precision_recall_fscore_support(y38v,val_pred,labels=np.arange(N38),average='macro',zero_division=0)

artifact={'pca_mean':pca38.mean_.astype(np.float32),'pca_components':pca38.components_[:k].astype(np.float32),'scaler_mean':scaler38.mean_[:k].astype(np.float32),'scaler_scale':scaler38.scale_[:k].ast
ype(np.float32),'model':model,'n_components':k,'classes':names38}
    artifact_path=OUT/f'{name}.joblib'; joblib.dump(artifact,artifact_path,compress=3); size_mb=artifact_path.stat().st_size/1_000_000
    for ii in lat_idx[:5]: one_image_predict(X38v_u8[ii],k,model)
    lats=[]
    for ii in lat_idx:
        t0=perf_counter_ns(); one_image_predict(X38v_u8[ii],k,model); lats.append((perf_counter_ns()-t0)/1_000_000)
    p50=float(np.percentile(lats,50)); p95=float(np.percentile(lats,95))

challenge_rows.append({'Configuration':name,'Training_labels':label_source,'Loss_or_model':kind,'PCA_components':k,'Validation_accuracy':acc,'Validation_macro_precision':pr,'Validation_macro_reca
ll':rc,'Validation_macro_F1':f1,'Classifier_fit_seconds':fit_s,'Validation_batch_prediction_seconds':batch_predict,'Pipeline_latency_mean_ms':float(np.mean(lats)),'Pipeline_latency_p50_ms':p50,'Pipeline
_latency_p95_ms':p95,'Saved_pipeline_MB_decimal':size_mb,'Meets_5MB_artifact_limit':size_mb<=5.0,'Meets_50ms_Colab_p95_proxy':p95<=50.0,'Convergence_warning':warn})
    saved_models[name]={'model':model,'artifact':artifact,'path':artifact_path,'k':k}
    print(name,'accuracy=',f'{acc:.4f}','MacroF1=',f'{f1:.4f}','fit=',f'{fit_s:.4f}','sizeMB=',f'{size_mb:.4f}','p50ms=',f'{p50:.4f}','p95ms=',f'{p95:.4f}')
    if warn: print('Convergence warning:',warn)
    del model,artifact; gc.collect()


challenge_df=pd.DataFrame(challenge_rows); challenge_df.to_csv(OUT/'task12_38class_adaptation_results.csv',index=False)
print(challenge_df[['Configuration','Validation_accuracy','Validation_macro_F1','Pipeline_latency_p50_ms','Pipeline_latency_p95_ms','Saved_pipeline_MB_decimal','Meets_5MB_artifact_limit','Meets_
50ms_Colab_p95_proxy']].round(4).to_string(index=False))


# Select the best noisy-label setting that meets the measured artifact and latency proxies.
eligible=challenge_df[(challenge_df.Training_labels=='noisy')&challenge_df.Meets_5MB_artifact_limit&challenge_df.Meets_50ms_Colab_p95_proxy]
if eligible.empty:
    print('No noisy-label configuration passed both measured proxy limits.')
else:
    chosen=eligible.sort_values('Validation_macro_F1',ascending=False).iloc[0]; name=chosen['Configuration']; mdl=saved_models[name]['model']; k=saved_models[name]['k']
    test_pred=mdl.predict(Z38t100[:,:k]); test_acc=accuracy_score(y38t,test_pred); _,_,test_f1,_=precision_recall_fscore_support(y38t,test_pred,labels=np.arange(N38),average='macro',zero_division=0)
    print('Selected:',name,'test_accuracy=',f'{test_acc:.4f}','test_macroF1=',f'{test_f1:.4f}')


# Entropy and majority-class baseline for train/validation.
for split, labels in [('Training',y38tr),('Validation',y38v)]:
    counts=np.bincount(labels,minlength=N38).astype(float); p=counts/counts.sum(); mask=p>0
    H=-np.sum(p[mask]*np.log2(p[mask])); Hmax=np.log2(N38); majority=int(np.argmax(counts))

print(split,'entropy_bits=',f'{H:.4f}','max_entropy_bits=',f'{Hmax:.4f}','normalised_entropy=',f'{H/Hmax:.4f}','majority_class=',names38[majority],'majority_accuracy=',f'{counts[majority]/counts.sum():.
4f}')


# End of consolidated reproducibility code.
```